



A Direct Data Aware LSTM Neural Network Architecture for Complete Remaining Trace and Runtime Prediction

(IEEE TRANSACTIONS ON SERVICES COMPUTING, 2023)

Mingyun Kang

alsrbs3345@pusan.ac.kr

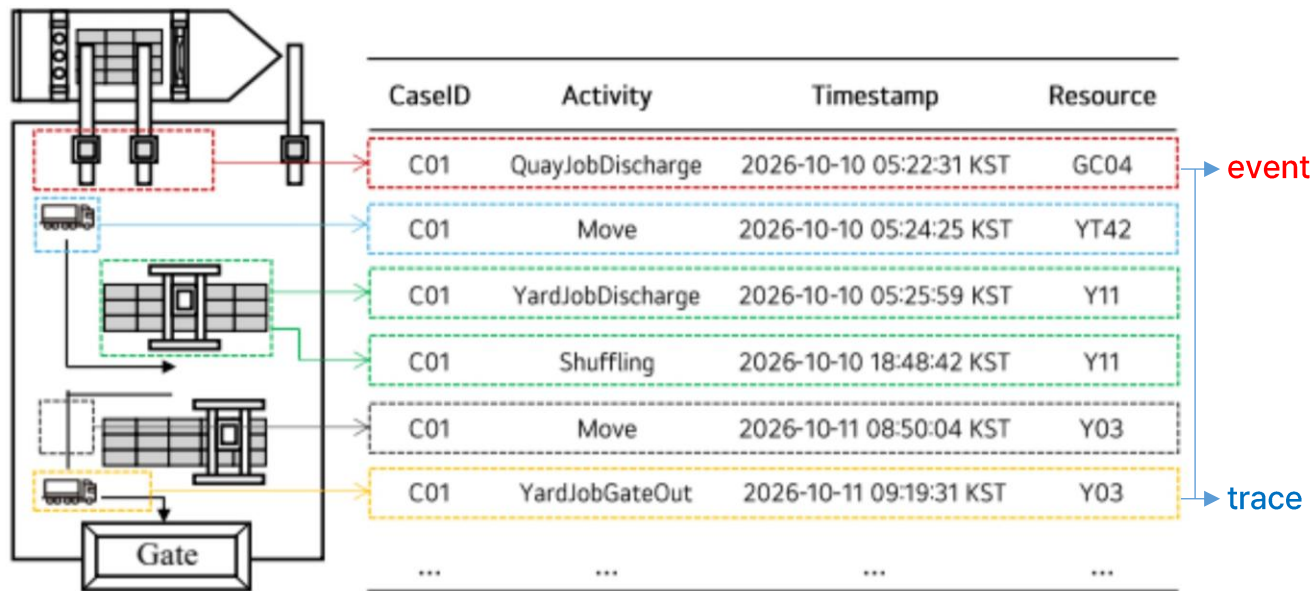
Contents

1. Background
2. Methodology
3. Methodology Code
4. Experiment
5. Limitation
6. Future work

Background

■ Process Mining

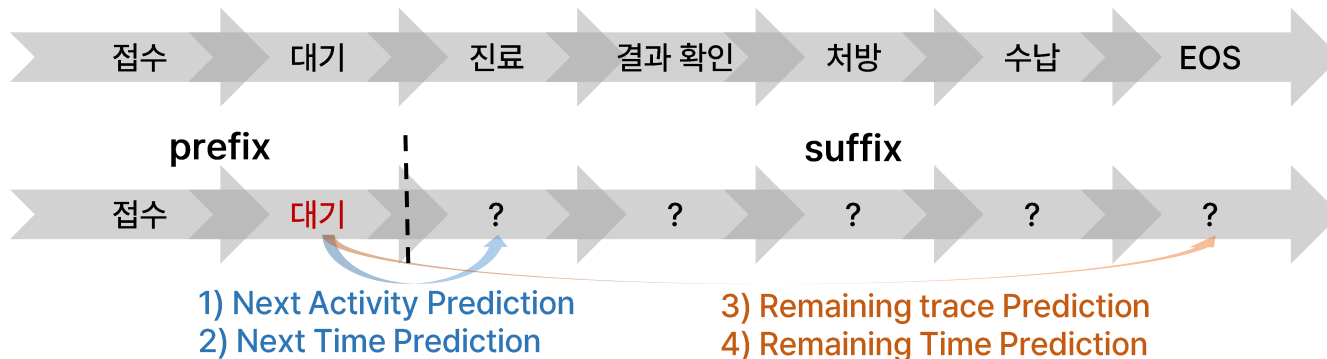
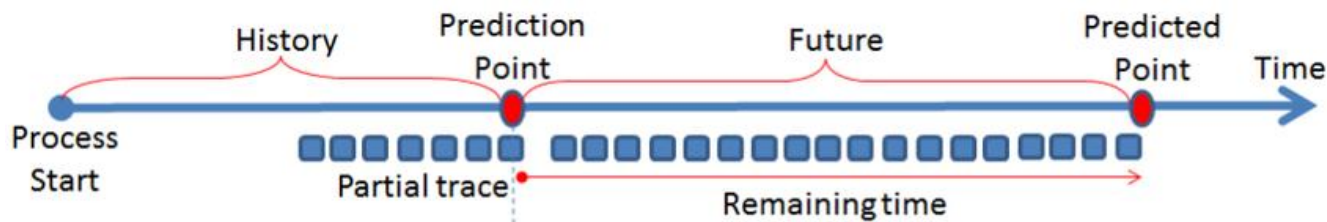
- 정의: Event Log의 실행 이력을 분석하여 업무 프로세스의 흐름과 특성을 파악하는 데이터 기반 방법론
- 핵심 구성요소: Case ID(프로세스 식별), Activity(활동), Timestamp(시간), Attribute(부가 정보)
- 이를 통해 프로세스 병목 구간 발견, 실행 패턴 분석 등에 활용



Background

▪ Predictive Process Monitoring (PPM)

- 진행 중인 프로세스의 향후 흐름과 소요 시간을 예측하는 기법
- 활동 예측: Next Activity Prediction(다음 활동) & Remaining Trace Prediction(남은 활동)
- 시간 예측: Next Time Prediction(다음 시간) & Remaining Time Prediction(남은 시간)



Background

▪ Predictive Process Monitoring (PPM)

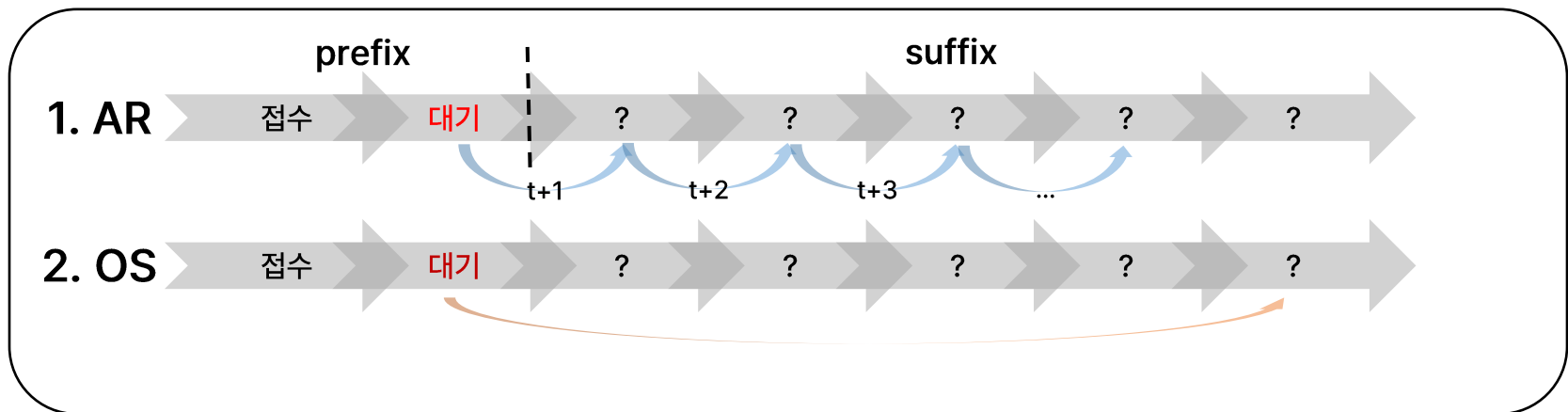
- PPM의 입력 데이터 구조와 분석 방식 비교
 - 필수: Case ID, Activity, Timestamp
 - 선택: Attribute (담당자, 비용 등 부가 정보)
- 분석 방식
 - Activity-only 방식: 활동 순서 정보만으로 예측(간단하지만 맥락 부족)
 - Data-aware 방식: 활동 순서 + Attribute 결합(풍부한 맥락)

Case ID	Activity	Timestamp	Case Attribute	Event Attribute
Loan-001	대출 신청 완료	09:00	신청 금액: 3,000만 원	채널: 온라인
Loan-001	신용 조회	09:20	신청 금액: 3,000만 원	담당자: 신용 평가원 A
Loan-001	소득 확인	10:10	신청 금액: 3,000만 원	담당자: 대출 심사원 B
Loan-001	?	?	신청 금액: 3,000만 원	?

Background

▪ Predictive Process Monitoring (PPM)

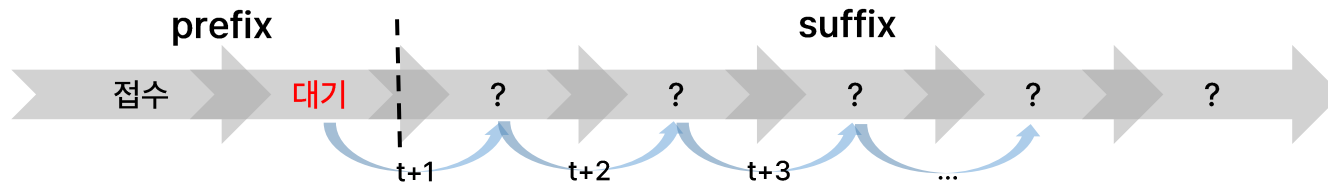
- 남은 trace를 예측하는 두 가지 방식
- 1. Autoregressive(AR) 방식
 - 다음 이벤트 하나를 예측하고, 그 예측값을 다시 입력으로 사용해 다음 이벤트를 반복적으로 예측하는 방식
- 2. One-shot 방식
 - 남은 trace 전체를 한 번에 예측하는 방식



Methodology

SEP-LSTM

- 기존 (LSTM 기반) PPM은 주로 Single Event Prediction, SEP 방식
 - 현재까지 관측된 값(Prefix)을 입력으로 다음 Activity, Timestamp를 예측
 - 예측된 event를 다시 입력에 추가하고, 종료 event(EOS, End-of-Sequence)가 나올 때까지 반복



- 이 방식은 전체 trace를 한 번에 예측하지 않고, 다음 event 예측을 반복하여 trace를 완성함

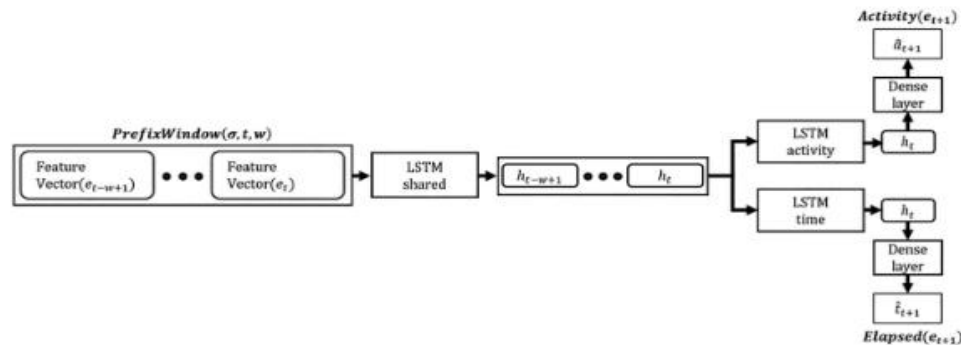


Fig. 1. SEP-LSTM: Main model components.

Methodology

SEP-LSTM 핵심 한계

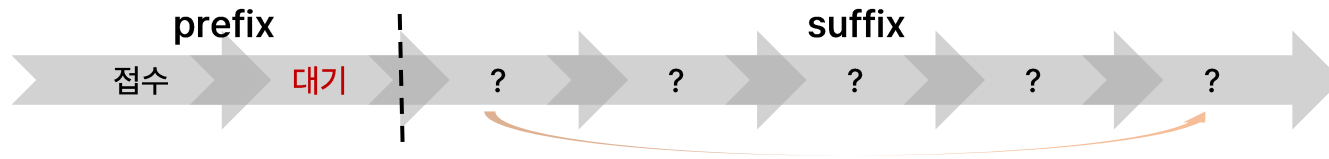
- 첫째, 예측값을 다시 입력에 사용하기 때문에 오차 누적(Exposure Bias)이 발생
- 둘째, Event attribute 활용이 어려움
 - Case attribute는 case 전체에서 값이 동일하므로 쉽게 사용 가능한데, Event attribute는 event마다 값이 달라짐
 - 미래 attribute에 대한 처리 문제를 발생시킴(예측해서 계속 사용하겠다 or Prefix에 있는 attribute만 사용하겠다)

Case ID	Activity	Timestamp	Case Attribute	Event Attribute
Loan-001	대출 신청 완료	09:00	신청 금액: 3,000만 원	채널: 온라인
Loan-001	신용 조회	09:20	신청 금액: 3,000만 원	담당자: 신용 평가원 A
Loan-001	소득 확인	10:10	신청 금액: 3,000만 원	담당자: 대출 심사원 B
Loan-001	?	?	신청 금액: 3,000만 원	?
Case ID	Activity	Timestamp	Case Attribute	Event Attribute
Loan-002	대출 신청 완료	09:25	신청 금액: 2,500만 원	채널: 오프라인
Loan-002	신용 조회	09:30	신청 금액: 2,500만 원	담당자: 신용 평가원 C
Loan-002	소득 확인	10:00	신청 금액: 2,500만 원	담당자: 대출 심사원 B
Loan-002	?	?	신청 금액: 2,500만 원	?

Methodology

▪ CRTP-LSTM

- Complete Remaining Trace Prediction, CRTP-LSTM를 제안
 - 관측된 Prefix만 입력으로 사용하여
 - 전체 activity sequence와 remaining runtime을 한 번에 예측
 - 예측된 event를 다시 입력하지 않기 때문에, 미래 event의 attribute를 처리할 필요 없음



SEP-LSTM

$[A, B, C] \rightarrow D$

$[A, B, C, \hat{D}] \rightarrow E$

$[A, B, C, \hat{D}, \hat{E}] \rightarrow EOS$

CRTP-LSTM

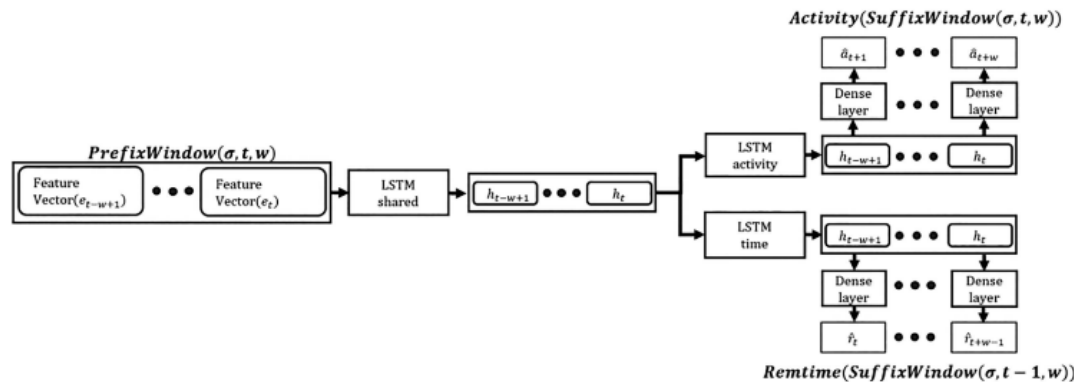
$[A, B, C] \rightarrow [D \rightarrow E \rightarrow EOS]$

t+2의 E를 예측하기 위해 t+1에서
예측한 D를 다시 input에 넣지 않음

Methodology

■ CRTP-LSTM

- Input: Prefix Feature Vectors: $x_t = [x_t^{act} || x_t^{time} || x_t^{attr}]$
 - 각 event의 activity, timestamp, attribute를 임베딩 한 후 하나의 feature vector로 결합
- Shared Bi-LSTM Layers
 - Prefix feature vector sequence를 양방향으로 처리
 - Activity와 runtime prediction에 공통으로 사용되는 representation 학습
- Task-specific Bi-LSTM
 - Shared representation을 Activity/Runtime branch로 분리
 - 각 prediction task에 특화된 representation 학습



Methodology

■ CRTP-LSTM

- Prefix-Suffix Training
 - Complete Trace $[A \rightarrow B \rightarrow C \rightarrow D \rightarrow E]$
 - 하나의 완료된 trace를 각 관측 시점에서 분할하여 여러 training sample 생성

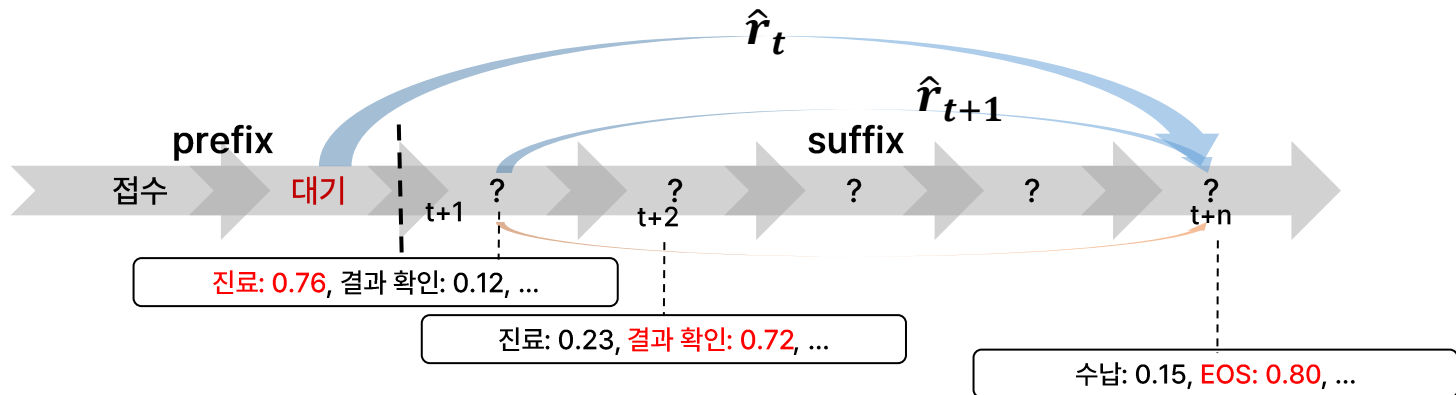
Observed Prefix	Activity Suffix	Runtime Suffix
A	$B \rightarrow C \rightarrow D \rightarrow E \rightarrow \text{EOS}$	$r_2, r_3, r_4, r_5, 0$
$A \rightarrow B$	$C \rightarrow D \rightarrow E \rightarrow \text{EOS}$	$r_3, r_4, r_5, 0$
$A \rightarrow B \rightarrow C$	$D \rightarrow E \rightarrow \text{EOS}$	$r_4, r_5, 0$
$A \rightarrow B \rightarrow C \rightarrow D$	$E \rightarrow \text{EOS}$	$r_5, 0$

- Observed Prefix를 입력으로, 이후의 전체 Activity Sequence와 각 시점에서의 Remaining Runtime을 정답으로 학습
- Activity Suffix 각 위치에 대해 Cross Entropy를 계산 $[B, C, D] \leftrightarrow [\hat{B}, \hat{C}, \hat{D}]$
- Runtime은 next-time을 누적해서 구하지 않고, 해당 시점에서 끝 시점의 시간을 바로 예측(MAE)

Methodology

■ CRTP-LSTM

- [1] 미래 각 시점의 activity probability distribution
 - 미래 각 시점마다 가능한 activity label의 확률분포를 출력
 - 각 시점에서 가장 확률이 높은 activity를 선택
 - EOS가 선택되면 해당 지점을 trace의 종료 시점으로 판단
- [2] 미래 각 시점의 remaining runtime prediction
 - 미래 각 시점마다 remaining runtime value를 예측
 - $\hat{r}_t$ 는 현재 prefix 시점에서 case 종료까지 남은 시간, $\hat{r}_{t+1}, \dots$ 는 미래 event 이후에 남아 있는 시간
 - 최종 remaining runtime은 $\hat{r}_t$



Methodology Code

```
class CRTPPrefixEncoder(nn.Module): # prefix의 Activity, Attribute, Timestamp를 잠재 벡터로 변환
    def __init__(
        self,
        activity_vocab_size: int, # Activity 종류 개수
        categorical_vocab_sizes: list[int], # Attribute별 category 개수 목록
        continuous_dim: int, # Timestamp 기반 수치 특징 개수
        activity_embed_dim: int = 32, # Activity를 표현할 임베딩 차원
        categorical_embed_dim: int = 16, # 각 Attribute를 표현할 임베딩 차원
        latent_dim: int = 128, # BiLSTM에 넣을 최종 잠재 벡터 차원
    ) -> None:
        super().__init__()

        self.activity_embedding = nn.Embedding( # Activity id를 학습 가능한 벡터로 변환
            activity_vocab_size,
            activity_embed_dim,
            padding_idx=PAD_ID,
        )

        self.categorical_embeddings = nn.ModuleList( # Resource, Role 같은 Attribute를 각각 임베딩
            [
                nn.Embedding(vocab_size, categorical_embed_dim, padding_idx=PAD_ID)
                for vocab_size in categorical_vocab_sizes
            ]
        )

        raw_dim = activity_embed_dim # Activity 임베딩 크기
        raw_dim += len(categorical_vocab_sizes) * categorical_embed_dim # 모든 Attribute 임베딩 크기
        raw_dim += continuous_dim # Timestamp 기반 수치 특징 크기

        self.to_latent = nn.Linear(raw_dim, latent_dim) # 결합된 prefix 이벤트 특징을 latent vector로 압축

    def forward(self, activity_x, categorical_x, continuous_x):
        parts = [self.activity_embedding(activity_x)] # 각 prefix 시점의 Activity를 벡터화

        for col_idx, embedding in enumerate(self.categorical_embeddings):
            parts.append(embedding(categorical_x[:, :, col_idx])) # 각 Attribute를 같은 시점의 벡터로 변환

        parts.append(continuous_x) # Timestamp에서 만든 수치 특징을 그대로 추가
        event_features = torch.cat(parts, dim=-1) # Activity, Attribute, Timestamp 특징을 하나로 결합

        return self.to_latent(event_features) # 각 prefix 시점을 BiLSTM 입력용 latent vector로 변환
```

■ CRTPPrefixEncoder

- BiLSTM에 들어가기 전, prefix를 잠재 벡터로 변환하는 단계
- Activity, Attribute를 임베딩 벡터로 변환
- Timestamp는 수치형 컬럼이라 임베딩 하지 않고 그대로 concat
- 각 시점 t마다 Activity | Timestamp | Attribute를 이어 붙이고 Linear 층을 거쳐 잠재 벡터로 변환함
- 잠재 벡터는 BiLSTM의 입력으로 들어가게 됨

Methodology Code

■ CRTPLSTM

- 잠재 벡터를 양방향 LSTM으로 학습
- 미래의 각 시점별로 가능한 activity들의 점수를 만들고 softmax를 통해 확률분포로 변환
- Remaining time 예측은 미래 시점별 남은 실행 시간을 출력

```
class CRTPLSTM(nn.Module): # complete remaining trace와 runtime을 예측하는 CRTP-LSTM
    def __init__(
        self,
        activity_vocab_size: int, # PAD, END를 포함한 활동 사전 크기
        categorical_vocab_sizes: list[int], # 추가 범주형 특징 사전 크기들
        continuous_dim: int, # 시간/수치형 특징 개수
        latent_dim: int = 128, # prefix 잠재 벡터 차원
        hidden_dim: int = 128, # BiLSTM 은닉 차원
        dropout: float = 0.2,
    ) -> None:
        super().__init__()

        self.encoder = CRTPPrefixEncoder( # prefix 이벤트를 잠재 벡터로 만드는 입력 인코더
            activity_vocab_size=activity_vocab_size,
            categorical_vocab_sizes=categorical_vocab_sizes,
            continuous_dim=continuous_dim,
            latent_dim=latent_dim,
        )
        self.dropout = nn.Dropout(dropout)

        self.shared_bilstm = nn.LSTM( # prefix 전체 문맥을 읽는 공통 BiLSTM
            input_size=latent_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True,
        )

        branch_dim = hidden_dim * 2 # 양방향 LSTM 출력 차원

        self.activity_bilstm = nn.LSTM( # 남은 활동 순서 예측 전용 BiLSTM
            input_size=branch_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True,
        )
        self.activity_head = nn.Linear(branch_dim, activity_vocab_size) # 미래 시점별 활동 점수 출력

        self.runtime_bilstm = nn.LSTM( # 남은 실행 시간 예측 전용 BiLSTM
            input_size=branch_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True,
        )
        self.runtime_head = nn.Sequential( # 미래 시점별 남은 실행 시간 출력
            nn.Linear(branch_dim, 1),
            nn.PReLU(),
        )

    def forward(self, activity_x, categorical_x, continuous_x):
        prefix_latent = self.encoder(activity_x, categorical_x, continuous_x) # prefix를 잠재 벡터로 변환
        prefix_latent = self.dropout(prefix_latent) # 잠재 벡터 regularization

        shared_states, _ = self.shared_bilstm(prefix_latent) # prefix 문맥을 공통 표현으로 인코딩

        activity_states, _ = self.activity_bilstm(shared_states) # 활동 예측 분기 표현
        activity_logits = self.activity_head(activity_states) # complete remaining trace 점수
        activity_probs = F.softmax(activity_logits, dim=-1) # 미래 시점별 활동 확률분포

        runtime_states, _ = self.runtime_bilstm(shared_states) # 실행 시간 예측 분기 표현
        runtime_pred = self.runtime_head(runtime_states).squeeze(-1) # 미래 시점별 남은 실행 시간

        return activity_logits, activity_probs, runtime_pred # CRTP의 두 예측 결과 반환
```

Methodology Code

▪ Loss 함수

- CRTP-LSTM의 loss는 activity loss, runtime loss로 두 가지 존재
- Activity loss는 Cross-Entropy를 사용
- Runtime loss는 MAE를 사용
- 최종 손실은 두 손실의 가중합

```
def crtp_loss(
    activity_logits: torch.Tensor, # 모델이 출력한 활동 점수
    runtime_pred: torch.Tensor, # 모델이 출력한 남은 실행 시간
    target_activity: torch.Tensor, # 정답 남은 활동 sequence
    target_runtime: torch.Tensor, # 정답 남은 실행 시간 sequence
    target_mask: torch.Tensor, # 실제 suffix 위치는 1, padding 위치는 0
    runtime_weight: float = 1.0, # 실행 시간 손실 가중치
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    activity_loss = F.cross_entropy( # 남은 활동 순서 예측 손실
        activity_logits.reshape(-1, activity_logits.size(-1)),
        target_activity.reshape(-1),
        ignore_index=PAD_ID,
    )

    runtime_error = torch.abs(runtime_pred - target_runtime) * target_mask # padding을 제외한 절대오차
    runtime_loss = runtime_error.sum() / target_mask.sum().clamp_min(1.0) # masked MAE

    total_loss = activity_loss + runtime_weight * runtime_loss # CRTP의 multi-task loss
    return total_loss, activity_loss, runtime_loss
```

Experiment

■ Experiment: Event logs

- 총 6개의 real-life event log를 활용해 모델 성능 비교
 - Variants: 서로 다른 activity sequence의 개수로, 높을수록 다양한 경로가 존재 (복잡도 상승)
 - Coverage: 상위 3개 trace variant가 차지하는 case 비율. 낮을수록 case들이 다양한 경로로 흩어져 있음
 - Structure: 가능한 direct-follow 관계 대비 process structure 제약 정도
 - Affinity: trace 간 평균 activity sequence 유사도(normalized Levenshtein similarity)
 - Variability(EF): trace 진행 중 event feature 값의 변화 정도
- BPIC20은 상대적으로 단순한 event log, BPIC12, BPIC17, BPIC19는 복잡한 event log
- BPIC19는 Variability(EF)가 0.95로 event feature가 trace 진행 중 95%로 바뀜

Event log	Sector	Cases	Variants	Avg. TL	Coverage	Structure	Affinity	Case feat.	Event feat.	Variability (EF)
BPIC20	University	10366	98	5.43	0.82	0.87	0.83	1	2	0.84
BPIC19	Retail	248455	11909	6.38	0.38	0.72	0.62	10	3	0.95
BPIC17	Finance	31509	4047	17.83	0.21	0.77	0.61	3	7	0.35
BPIC12	Finance	13087	4366	20.04	0.43	0.79	0.36	1	1	0.38
BPIC12-Sub	Finance	13087	1309	11.95	0.49	0.81	0.46	1	1	0.41
BAG	Operations	432357	7826	5.29	0.35	0.83	0.46	14	2	0.54

Experiment

■ Experiment: Event logs

Variant: 서로 다른 activity sequence의 개수

Trace A: [A→B→C→D→E] * 30개

Trace B: [A→B→C→E→D] * 25개

Trace C: [A→B→B→E→D] * 15개

Trace D: [A→C→E→D] * 10개

Trace E: [A→B→E→D] * 5개

다음과 같은 경우 variant의 개수는 5개가 됨. 전체 case 수 대비 variant의 개수가 적을수록, event log는 trace의 다양성이 적고 어느 정도 정형화된 프로세스를 가진다고 볼 수 있음

Event log	Sector	Cases	Variants	Avg. TL	Coverage	Structure	Affinity	Case feat.	Event feat.	Variability (EF)
BPIC20	University	10366	98	5.43	0.82	0.87	0.83	1	2	0.84
BPIC19	Retail	248455	11909	6.38	0.38	0.72	0.62	10	3	0.95
BPIC17	Finance	31509	4047	17.83	0.21	0.77	0.61	3	7	0.35
BPIC12	Finance	13087	4366	20.04	0.43	0.79	0.36	1	1	0.38
BPIC12-Sub	Finance	13087	1309	11.95	0.49	0.81	0.46	1	1	0.41
BAG	Operations	432357	7826	5.29	0.35	0.83	0.46	14	2	0.54

Experiment

Experiment: Event logs

Coverage: 상위 3개 trace variant가 차지하는 case 비율

Trace A: [A→B→C→D→E] * 30개

Trace B: [A→B→C→E→D] * 25개

Trace C: [A→B→B→E→D] * 15개

Trace D: [A→C→E→D] * 10개

Trace E: [A→B→E→D] * 5개

이 경우 상위 3개의 trace가 차지하는 case는 $30 + 25 + 15 = 70$ 개, 전체 case는 85개.
Coverage는 약 82.4%로 대부분의 case가 소수의 대표 trace variant에 집중되어, 정형화된 event log로 볼 수 있음

Event log	Sector	Cases	Variants	Avg. TL	Coverage	Structure	Affinity	Case feat.	Event feat.	Variability (EF)
BPIC20	University	10366	98	5.43	0.82	0.87	0.83	1	2	0.84
BPIC19	Retail	248455	11909	6.38	0.38	0.72	0.62	10	3	0.95
BPIC17	Finance	31509	4047	17.83	0.21	0.77	0.61	3	7	0.35
BPIC12	Finance	13087	4366	20.04	0.43	0.79	0.36	1	1	0.38
BPIC12-Sub	Finance	13087	1309	11.95	0.49	0.81	0.46	1	1	0.41
BAG	Operations	432357	7826	5.29	0.35	0.83	0.46	14	2	0.54

Experiment

■ Experiment: Compared Models

• 모델 구조

- TS: Transition System baseline
- SEP-LSTM: 기존 single event prediction 방식
- CRTP-LSTM: 논문이 제안한 complete remaining trace prediction 방식

• 입력 정보 수준

- Time-aware: Activity + Timestamp
- Data-aware: CF: Activity + Timestamp + Case attribute
- Data-aware: CF+EF: Activity + Timestamp + Case attribute + Event attribute

<u>Baseline</u>	<u>Time aware</u>	<u>Data aware: CF</u>	<u>Data aware: CF + EF</u>
TS($w = 1, k = 1$)	SEP-LSTM ($k = 1$)	SEP-LSTM ($k = 1$)	SEP-LSTM ($k = 1$)
TS($w = 1, k = 3$)	SEP-LSTM ($k = 3$)	SEP-LSTM ($k = 3$)	SEP-LSTM ($k = 3$)
TS($w = 2, k = 1$)	CRTP-LSTM	CRTP-LSTM	CRTP-LSTM
TS($w = 2, k = 3$)	-	-	-

Experiment

■ Evaluation Metric

- 기존 PPM 연구에서는 Damerau-Levenshtein을 많이 사용. 해당 지표는 인접 activity의 순서 교환을 허용하는데, 저자는 이것이 실제 병렬 실행을 과도하게 허용할 수 있다고 판단하여 더 엄격한 지표를 사용
 - Actual Trace: [진료 → 결과 확인 → 처방 → 수납], Predicted Trace: [진료 → 처방 → 결과 확인 → 수납]
- Normalized Levenshtein Similarity
 - $\text{Sim}_{lev}(X, Y) = 1 - \frac{d_{lev}(x,y)}{\max(|x|, |y|)}$, $d_{lev}(x,y) = \min\{\text{Insertion, Deletion, Substitution 횟수}\}$
 - Levenshtein: $d = 2$, $\text{Sim} = 0.50$
 - Damerau-Levenshtein: Swap 1회, $\text{Sim} = 0.75$
- 모델이 결과 확인과 처방의 순서를 반대로 예측했다면, Damerau-Levenshtein은 이를 단순한 Swap 한 번으로 평가함. 이러한 Swap은 두 Activity가 실제 병렬 관계라면 타당할 수 있지만, 모든 인접 activity의 순서 교환을 허용하면 프로세스 순서상 의미가 다른 예측도 관대하게 평가하게 됨. 이를 지적하며 더 엄격한 Levenshtein을 평가 지표로 사용

Experiment

Results: Remaining Trace Prediction

- Case + Event Feature CRTP-LSTM이 6개 로그 중 5개에서 최고 성능
- 복잡한 trace structure에서 CRTP이 확실히 우위를 가짐
 - BPIC19: SEP 0.3283 → CRTP 0.7499
- Event feature 활용 시 CRTP 성능 향상
 - Direct prediction으로 hallucinated event attribute 구성 불필요

	BPIC12	BPIC12-Sub	BPIC17	BPIC19	BPIC20	BAG
Baseline						
TS($w = 1, k = 1$)	0.0811	0.3284	0.1642	0.6758	0.9335	0.2997
TS($w = 1, k = 3$)	0.4020	0.4426	0.5256	0.6390	0.9335	0.6172
TS($w = 2, k = 1$)	0.3849	0.4419	0.1649	0.7074	0.9335	0.6723
TS($w = 2, k = 3$)	0.4803	0.5538	0.5217	0.7074	0.9335	0.6493
Time aware						
SEP-LSTM ($k = 1$)	0.3060	0.3619	0.6194	0.6999	0.9303	0.7215
SEP-LSTM ($k = 3$)	0.4258	0.5101	0.6315	0.6972	0.9195	0.7260
CRTP-LSTM	0.5436	0.6161	0.6078	0.6925	0.9282	0.7273
Data aware: CF						
SEP-LSTM ($k = 1$)	0.4792	0.5955	0.6202	0.7237	0.9414	0.9015
SEP-LSTM ($k = 3$)	0.4913	0.6109	0.6363	0.7281	0.9383	0.9049
CRTP-LSTM	0.5446	0.6341	0.6149	0.7485	0.9406	0.8935
Data aware: CF + EF						
SEP-LSTM ($k = 1$)	0.4510	0.5265	0.5187	0.3209	0.6644	0.9057
SEP-LSTM ($k = 3$)	0.4716	0.5372	0.5313	0.3283	0.6728	0.9074
CRTP-LSTM	0.5496	0.6439	0.6906	0.7499	0.9419	0.9037

Experiment

Results: Remaining Runtime Prediction

- MAE, RMSE ↓
 - 실제 remaining runtime과 예측 runtime의 오차. 낮을수록 좋은 성능
- Case + Event Feature CRTP-LSTM은 MAE 기준 모든 로그에서 최고 성능
- Attribute를 함께 활용할 때 성능 개선이 뚜렷함

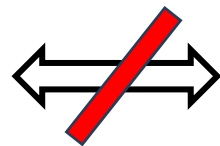
	BPIC12		BPIC12-Sub		BPIC17		BPIC19		BPIC20		BAG (sec.)	
Baseline	<u>MAE</u>	<u>RMSE</u>	<u>MAE</u>	<u>RMSE</u>	<u>MAE</u>	<u>RMSE</u>	<u>MAE</u>	<u>RMSE</u>	<u>MAE</u>	<u>RMSE</u>	<u>MAE</u>	<u>RMSE</u>
TS($w = 1, k = 1$)	8889	10005	10069	11214	12788	14387	47838	52125	7398	8168	980	1164
TS($w = 1, k = 3$)	8889	10005	10069	11214	12788	14387	47838	52125	7398	8168	980	1164
TS($w = 2, k = 1$)	8898	10013	10231	11414	12223	14325	47798	52392	7216	8033	963	1154
TS($w = 2, k = 3$)	8898	10013	10231	11414	12733	14325	47798	52392	7216	8033	963	1154
Time aware												
SEP-LSTM ($k = 1$)	13249	19284	10123	11433	12153	13548	45588	51034	6391	7519	605	789
SEP-LSTM ($k = 3$)	8986	12379	6428	8237	12621	14100	45826	51279	6412	7553	585	760
CRTP-LSTM	5248	6582	5796	7330	11667	13090	41127	43843	6356	7255	570	677
Data aware - CF												
SEP-LSTM ($k = 1$)	13964	18659	5784	6990	12095	13384	39965	43820	6362	7459	501	675
SEP-LSTM ($k = 3$)	15846	23195	5750	6908	12802	14207	39971	44102	6360	7459	461	613
CRTP-LSTM	5239	6445	5748	7138	11739	13097	40076	42107	6310	7220	550	654
Data aware - CF + EF												
SEP-LSTM ($k = 1$)	10561	16808	5850	7423	15434	20464	72390	90789	13045	18168	310	408
SEP-LSTM ($k = 3$)	9956	13730	5734	7220	14991	19778	71591	89672	12944	17984	299	393
CRTP-LSTM	5214	6581	5579	6986	10766	12750	38929	40624	6128	7050	293	349

Limitation

■ 첫째, Intra-case feature 중심

- 기존 LSTM 기반 PPM 연구, 이 논문의 CRTP-LSTM 역시 intra-case feature에 초점을 맞춤
 - 한 case 내부에서 관측된 activity, timestamp, attribute를 사용하지만, 동시에 진행 중인 다른 case들의 상태는 충분히 반영하지 못함
 - 예를 들어 대출 심사 프로세스라면, 특정 신청 건의 처리 시간은 그 신청 건의 금액이나 activity sequence만으로 결정되지 않을 수 있음.
 - 같은 시점에 접수된 신청 건 수, 담당 심사자의 업무량 등 동시에 진행 중인 case와의 경쟁 상황을 반영하지 못함

Case ID	Activity	Timestamp	Event Attribute
Loan-001	대출 신청 완료	09:00	채널: 온라인
Loan-001	신용 조회	09:20	담당자: 신용 평가원 A
Loan-001	소득 확인	10:10	담당자: 대출 심사원 B
Loan-001	?	?	?

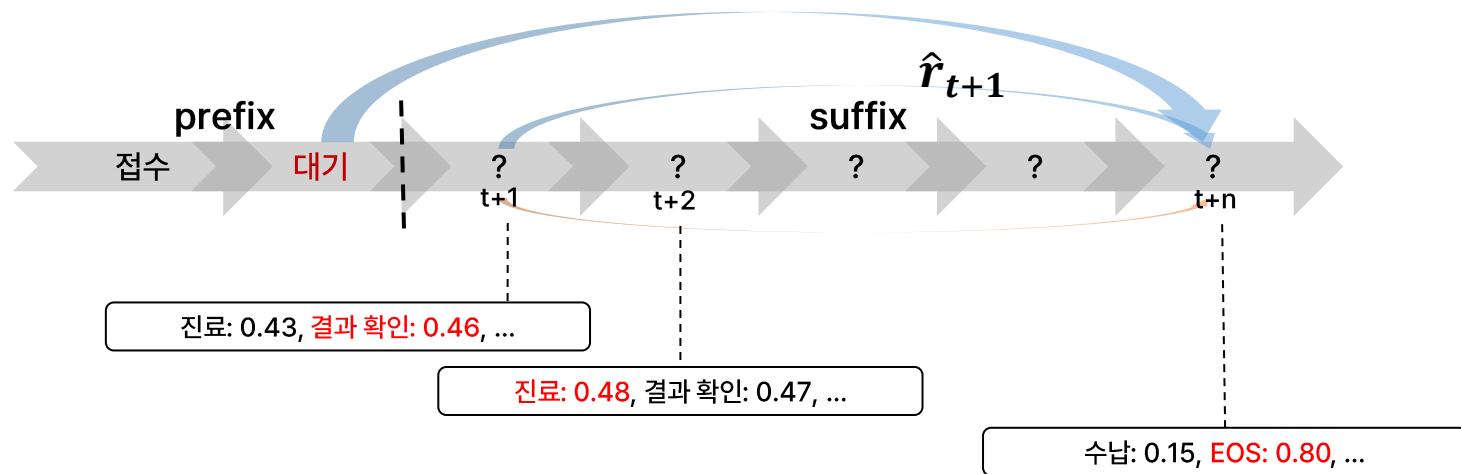


Case ID	Activity	Timestamp	Event Attribute
Loan-002	대출 신청 완료	09:04	채널: 온라인
Loan-002	신용 조회	09:23	담당자: 신용 평가원 A
Loan-002	?	?	?
Loan-001	?	?	?



Limitation

- 둘째, Activity 사이의 순차적 제약을 보장하지 않음
 - CRTP-LSTM은 미래 각 시점마다 activity label의 확률분포를 출력하고, 각 시점에서 가장 확률이 높은 activity를 선택
 - 각 시점만 보면 모두 그럴듯한 activity지만, 전체 trace를 보면 어느 하나의 process path에도 속하지 않는 trace가 될 수 있음
 - 평가 지표가 trace 유사도 중심이라서 예측 trace가 process constraint를 만족하는지, 실제 실행 가능한 trace인지에 대한 평가는 제한적임



Future work

■ 첫째, Inter-case feature 확장

- CRTP-LSTM뿐만 아니라, 대부분의 PPM 모델은 독립적인 case를 가정하여 예측을 수행함
- 개별 case 중심 예측에서 벗어나, 동시 진행 case들의 상호작용을 반영하는 inter-case aware model로 확장
- Senderovich et al. (2019)은 bi-dimensional encoding을 제안하여 case들의 상호작용을 반영
 - Intra-case dimension: 현재 case의 실행 이력
 - Inter-case dimension: 동시에 실행 중인 다른 case들과의 관계

■ 둘째, Process Constraint-aware Decoding

- CRTP-LSTM은 각 미래 시점의 activity distribution에서 가장 높은 확률의 label을 선택
 - Position-wise argmax는 전체 trace가 실제 process constraint를 만족하는지 보장하지 않음
 - 불가능한 activity transition 차단, 관찰 빈도가 낮은 transition penalty 부여 등의 방법으로 단순 확률적 선택의 한계를 보완
 - 가장 확률 높은 activity sequence가 아니라, 확률적으로 그럴듯하면서도 process상 가능한 remaining trace를 예측